

Product & System Architecture

Student OS Documentation — File 03

Architecture Goal

Use a modular full-stack architecture that can scale from a student MVP to a SaaS product without rewriting core modules.

Recommended Layers

Frontend: React-based application with reusable UI components and route-level pages.

API layer: RESTful or equivalent typed service layer between frontend and backend.

Backend: Node.js service with modular domain logic.

Database: relational database preferred for structured academic relationships.

Authentication: secure session or token-based authentication with server-side authorization.

File storage: separate object/file storage abstraction for note attachments.

Notification service: centralized reminder/notification scheduler abstraction.

AI service: isolated service/module so AI providers can be changed without rewriting product logic.

Domain Modules

Auth & Users

Dashboard

Timetable

Calendar

Tasks

Attendance

Exams

Notes

Notifications

Analytics

AI

Settings

Data Flow

User action → frontend validation → API request → authentication/authorization → domain service → database/service → normalized response → UI state update → related modules refresh.

Cross-module events should update dependent views without duplicated business logic.

Scalability

Keep domain logic separate from controllers.

Use pagination for growing collections.

Use indexes for frequently queried fields.

Centralize error handling and logging.

Keep third-party providers behind adapters/interfaces.

Architecture Rules

No business-critical logic only in the client.

No secrets in frontend code.

No direct database access from UI components.

Prefer typed request/response contracts.

Document major architectural decisions.